

0.66.3. La lettre bouge avec la croix — en simple

Le 0.66 en une ligne : `deplace_croix(x, y, ALPHABET[0], 133)`. Plus d'attente ni d'effacer à écrire.

1. Ce qu'il faut comprendre

C'est le 0.66, en plus simple : tout le bloc de la croix (l'attente, l'effacement, les quatre `if`, le `poser`) est remplacé par une ligne, `deplace_croix` (le 0.71).

La vitesse, 133 ms : c'est 8 images, exactement l'`attente = 8` du 0.66. La lettre va donc à la même vitesse.

Et elle ne clignote plus : `deplace_croix` n'efface l'ancienne case que si la lettre a bougé. Le 0.66 effaçait et reposait à chaque pas possible.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1748 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 9;           // la lettre part du milieu de l'écran
```

2. UNE VARIABLE — Y

```
uint8_t y = 8;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

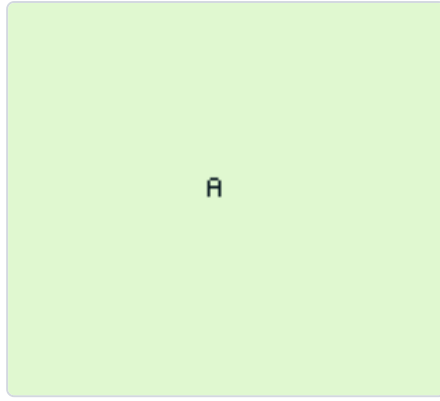
```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 133); // tout le bloc du 0.66 : 133 ms = 8 images
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Comme au 0.66 : un A au milieu, qui se déplace avec les flèches.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la croix le déplace

✓ il est à sa place

Ensuite : **0.67. Les variables de la lettre dans leur propre fichier** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++